# String Matching Algorithms

String matching algorithms search for occurences of a pattern string P of length m with a text string T of length n ($m \leq n$).

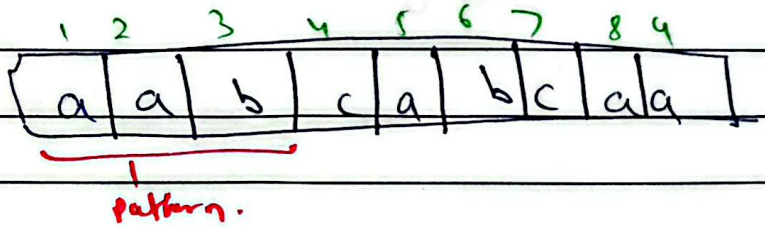
e.g

T = aabcabcaa , n = 9

P = aab , m = 3

result, validshift = 0.

↳ The value of the index(es) where pattern matches but one before it.




So valid shift = 1-1 = 0

consider if P was cab

then valid shift = 3

BHATTI

CamScanner

/date

. valid shifts can be more than one.

. we are considering overlapping pattern as well.

e.g T = 1 2 3 4 5 6 7 8 9
a b a b a b a c a b

P = a b a

now There are 2 valid shifts one at 0 & other at 2.

valid shift is pseudo index from a string.

* m = length of Pattern
* n = length of Text

BHATTI

CamScanner

/date

We are going to learn 4 string matching algos:

1) Naive
2) Rabin - Karp
3) Finite Automaton
4) KMP (Knuth - morris - Pratt)

## 1) Naive algorithm:

This checks every possible starting position (called a shift) in text to see if pattern matches.

Pseudocode

```
NAIVE-string-Matcher (T, P, n, m)

    for  s = 0  to  n-m
        j = 1
        while  j <= m and P[j] = T[s+j]
            j = j+1
        if  j = m+1
            print "Pattern occured shift: " + s
```

BHATTI

CamScanner

date

Example:

T = acaabc , n=6

P = aab , m=3.

| Shift (s) | P[1-3] | T[s+1 --- s+m] (s+3) | |
|---|---|---|---|
| 0 | aab | aca | ✗ |
| 1 | aab | caa | ✗ |
| 2 | aab | aab | ✓ Pattern Occurs shift: 2 |
| 3 | aab | abc | ✗ |

Complexity:

outer loop goes s=0 to n-m

so (n-m+1)

& inner loop runs m times so

$\Theta(n-m+1) * \Theta(m)$.

BHATTI

CamScanner

/date

worst case time complexity: $\Theta(n-m+1) \cdot m$.

Why is it bad?

This is not a good algo at all because whenever a mismatch occurs it forgets all the previously gained info called content.

BHATTI

CamScanner

/date

## 2) Rabin Karp

It can speed up string matching by comparing hash values instead of individual characters at every shift.

Instead of character comparisons, Rabin Karp computes a numerical hash of the pattern then scans T using a rolling hash to update the text window hash in O(1) constant time per shift

In rolling hash, we donot need to recompute the hash of whole text window instead we can remove the first character & add new one. we will see that in Pseudocode.

BHATTI

CamScanner

y/date

Psuedocode:

```
Rabin-Karp-Matcher (T, P, d, q)
    # d = base of alphabets we using
    # e.g 0,1,2,3,4,5,6,7,8,9 , d = 10.
    #       0,1,2,3 , d = 4.

    # q = a large prime number.

    n = T.length
    m = P.length

    h = (d^(m-1)) mod q

    p = 0  # pattern hash value.
    t0 = 0 # Text window hash value. (intial)

    # Now find p & t0

    for i = 1 to m
        p = (d * p + P[i]) mod q
        t0 = (d * t0 + T[i]) mod q.
```

BHATTI

CamScanner

y/date

# now match loop to find patterns:

for s = 0 to n-m

if p == ts then
# hash matches, now check if characters
# match too, to handle
# spurious hits.

if P[1...m] == T[s+1...s+m] then

print "Valid shift : " + s.

# Now hash for next window

if s < n-m then

$$t_{s+1} = \left( d(t_s - T[s+1]*h) + T[s+m+1] \right) \bmod q$$

if $t_{s+1} < 0$ then

$t_{s+1} = t_{s+1} + q$

# if $t_{s+1}$ comes negative add
q into it to make it
a positive number.

BHATTI

CamScanner

ate

## Understanding the algo:

**d:**
This is basically the base of our numbers. / Total number of elements in our alphabet.

**q:**
a large prime number
# In hashing we prefer prime number, bc2 they are less prone to leading to spurious hashes.

**Spurious hashes:**
2 pattern that are not same get same hashes
To prevent this we use q of a large prime number.

**ts:**
hash of the particular text window. e.g if m = 4
$t_0$ = [1..4] hash , s = 0
$t_1$ = hash of [2...5] , s = 1
$t_2$ = hash of [3..6] , s = 2

BHATTI

CamScanner

date

How does it find hashes?

e.g consider base – 10 & pattern:

P = 12451 , m = 5

T = 123 12451 29 , n = 10

q = 13 , d = 10

To see hashing of Pattern we will do:

$p = (d * p + P[i]) \bmod q$

→ for all m times. = 5.

P[1] = 0.

i = 01: $p = (10 * 0 + P[1]) \bmod 13$

$p = 1$

i = 2: $p = (10 * 1 + P[2]) \bmod 13$

$p = 12$

i = 3: $p = (10 * 12 + P[3]) \bmod 13$

$p = 7$

BHATTI

CamScanner

e

i = 4: $P = (10 * 7 + P[4]) \bmod 13$

$P = 10$

i = 5: $P = (10 * 10 + P[5]) \bmod 13$

$P = 10$

so final hash = 10

$P = 10$

An easier way: (only for paper).

Because of modular arithmetic rules:

$(a+b) \bmod q = ((a \bmod q) + (b \bmod q)) \bmod q$.

$(a \cdot b) \bmod q = ((a \bmod q) \cdot (b \bmod q)) \bmod q$.

- instead of applying all these many steps we can simply do:

Hash = Pattern % q.

In previous example.

Hash = (12451) % 13

P = 10

easy peezy!!

BHATTI

CamScanner

What if negative hashes come?

In this case at end we can see we added q into the hash to make it positive.

Why don't we need to re calculate full hash for text window?

For this if you remember we calculated h
# let's forget modular arthemetic for now

consider T = 2 3 5 2 1 9 8 2 , d = 10
1 2 3 4 5 6 7 8

consider s = 2 so shift = 52198
we need to find $t_{s+1}$ = 21982

so we need to remove our MSB & add our LSB.

$h = d^{m-1} = 10^{5-1} = 10000$

(we are ignoring modular arthemetic currently).

~~always we know our h is now~~

BHATTI

y/date

Then see the last condition in code (ignore mod) & see we multiply h with T[s+1] so

$h = 10000 * 5$

$= 50000$

$52198 - 50000 = 2198.$

msb removed

how to add LSB & multiply with base 10 & add.

$2198 * 10 +$ new number $(T[s+m+1])$

$21980 + 2$

$\boxed{21982.}$

easy.

But all this is done with modular arthemetic bcz computer cannot store huge numbers.

I did this example without mod for understanding & this is exactly what happens any how too.

BHATTI

CamScanner

//date

Example:

What if not base 10?

consider we are given base 2.

P = 00 110

T = 00110 11001 00011 00011111

our q will also be in base 2

eg 0111 & our d = $10_{(2)}$ (base 2 for 2)

still now nothing change it will work exactly like base 10, just numbers will change.

For ease in Paper we can convert to base 10:

P = 00 110 = 6

q = 0111 = 7.

$t_0$ = (00110) % 7 = 6 % 7 = 6.

p = 6 % 7 = 6 ,

$t_1$ = (0 110 1) % 7 = 13 % 7 = 6

$t_2$ = (11010) % (0111) = 26 % 7 = 5

BHATTI

CamScanner

late

This makes it easier.

What if number are not given & pattern in alphabets?

e.g $P = a, b, c, d.$ $\Sigma = \{a, b, c, d, e\}$

↳ alphabet (all letters in our dictionary).

In this case simply assign each character in the alphabet a number & use those

$\Sigma = \{1, 2, 3, 4, 5\}$
$\quad\quad a \quad b \quad c \quad d \quad e$

$P = 1\ 2\ 3\ 4$

$d = box\ 5 = 5$

BHATTI

CamScanner

day/date

Full example:

T = { 2 3 5 9 0 2 3 1 4 1 5 2 6 7 3 9 9 2 1 }, n = 9

P = { 3 1 4 1 5 }, m = 5.

d = 10 as base 10

q = 13

Start:

$h = 10^4 \bmod 13 = 3$

finding p & to:

$p = (10 * 0 + 3) \bmod 13 = 3$
$p = (10 * 3 + 1) \bmod 13 = 5$
$p = (10 * 5 + 4) \bmod 13 = 2$
$p = (10 * 2 + 1) \bmod 13 = 8$
$p = (10 * 8 + 5) \bmod 13 = 7$

so $p = 7$

to window = { 2 3 5 9 0 }
using easy way to calculate its hash.
= 23590 % 13 = 8

BHATTI

CamScanner

ate

no

if p == ts
p == h0
7 == 8
✗ not true.
so find next hash:

now using the code:

$$t_{s+1} = \left( d(t_s - T[s+1] * h) + T[s+m+1] \right) \bmod q.$$

$$= \left(10(8 - 2*3) + 2\right) \bmod 13$$

$$= 9.$$

$$t_{s+1} = \left( d(t_s - \underbrace{T[s+1]}_{\text{MSB digit to remove}} * h) + \underbrace{T[s+m+1]}_{\text{new LSB to add}} \right) \bmod q.$$

now keep on doing this till we get our shifts.

BHATTI

CamScanner

/date

Complexity:

Preprocessing = $\Theta(m)$

Average case = $\Theta(n+m)$.

Worst case = $\Theta((n-m+1)\cdot m)$

↳ happen when the whole text, every shift matches pattern e.g

T = AAAAAAA

P = AAA.

BHATTI

CamScanner

## 3) Finite Automaton string matching:

This constructs a deterministic finite state machine during preprocessing that processes text characters one by one in a single pass.

* abstract machine
* Finite state
* Change from one state to another is called a transition.

Machine parts:

$$M(Q, q_0, A, \Sigma, \delta)$$

$Q$ = Finite set of states
$q_0$ = initial state
$A$ = final accepting state.
$\Sigma$ = Language / Alphabets.
$\delta$ = transition functions

BHATTI

CamScanner

/date

$\delta$ (current state, character) = new state.

If on current state & we receive character then go to this new state.

Suffix function:

To find the new state, we need to understand suffix function.

e.g if it does not match, should we start from start or go to the state 1 or 2 keeping content in mind.

For this content keeping we have suffix function.

Suffix function:

longest length where prefix == suffix.

BHATTI

CamScanner

/date

e.g

aba.

now if we consider prefix of ba, is a & suffix of ~~aba~~ ab is a

| Prefixes | | suffixes |
|---|---|---|
| a | ✓ matches | a , length = 1 |
| ab | ✗ not match | ba |

so for the suffix function return 1.

But in reality it actually works by dividing the pattern : a ; ab , aba & then finding ~~out~~ length of ~~greatest~~ greatest matching in all 3.

BHATTI

CamScanner

'/date

e.g:

a b c a b

| length of string | Prefixes | Suffixes | longest equal length |
|---|---|---|---|
| 1-(a) | no prefix, 0 | no suffix → 0 | 0 |
| 2-(a b) | a (no same) — a is prefix of b | b — b is suffix of a. | 0 |
| 3-abc | a, ab (no same) | c, bc | 0 |
| 4-abca | a (same), ab, abc | a (same), ca, bca | 1 — same have length of 1. |
| 5-abcab. | a, ab (same), abc, abca | b, ab (same), cab, bcab | 2 — ab = length 2. |

So at end suffix function returns 2.

BHATTI

CamScanner

'date

Now as we have understood Suffix function, let's make transition table.

Transition table:

consider

P = ababaca , m = 7

T = abababacaba , n = 11

Σ = {a, b, c}

Now for table:

Total States/rows = m+1

Total columns = no. of letters in language = 3+1 = 4

| q | a | b | c | rough work for understand |
|---|---|---|---|---|
| This will have state number of row for 0. | from q x if we get next character a which state to go next? | same as 0 but character is b | same as 0 but char is c. | Here write the suffix function result for each character is added. |

write suffix function result matching only write length

BHATTI

States start from 0.

| q | a | b | c | | | |
|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 0 | a0 | b0 | c0 |
| | | | | | suffix returns 0 | suffix returns 0 |

P= abaabaca.

first is a so it get a goto next state. which is 1.

| q | a | b | c | | | |
|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 0 | a0 | b0 | c0 |
| 1 | 1 | 2 | 0 | aa | ab | ac |

new characters added in path

suffix returns 1

suffix gives 0.

P= ab a ---

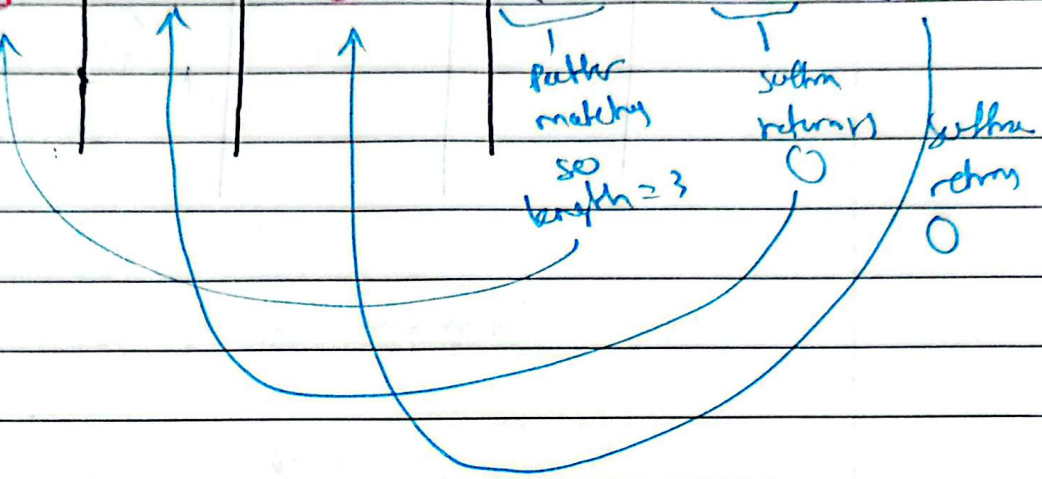
it is was b so gets length of pattern.

BHATTI

CamScanner

y/date

| q | a | b | c | | | |
|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 0 | a0 | b0 | c0 |
| 1 | 1 | 2 | 0 | aa | ab | ac |
| 2 | 3 | 0 | 0 | aba | abb | abc |




Like this complete whole table

| q | a | b | c | | | |
|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 0 | a0 | b0 | c0 |
| 1 | 1 | 2 | 0 | aa | ab | a |
| 2 | 3 | 0 | 0 | aba | abb | abc |
| 3 | 1 | 4 | 0 | abaa | abab | abac |
| 4 | 5 | 0 | 0 | ababa | ababb | ababc |
| 5 | 1 | 4 | 6 | abababa | ababab | ababac |
| 6 | 7 | 0 | 0 | ababaca | ababacb | ababacc |
| 7 | 1 | 2 | 0 | abacaa | abacab | abacac |

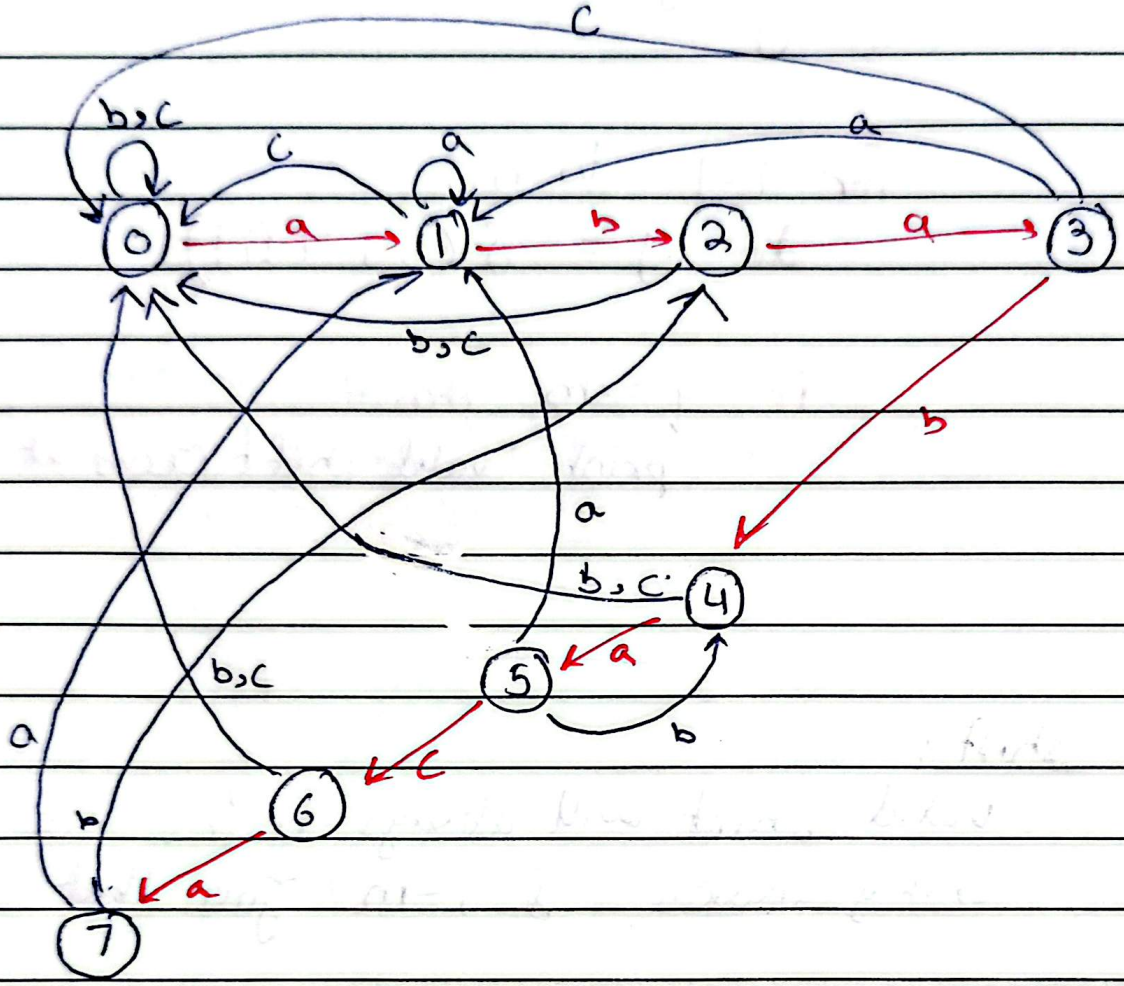
BHATTI

CamScanner

lay/date

## Now make FA: using transition table

Basically values in a, b, c columns represent next state.




TIP: If you know how to make FA from Theory of Automata course just make it & then replicate to the transition table. Prone to mistake but quite fast.

BHATTI

CamScanner

day/date

Pseudocode:

```
FA-Matcher (T, δ, m)

    n = T.length
    q = 0

    for i = 1 to n
        do q = δ(q, T[i])

        if q == m then # if we reach final state
            print "valid shift occurs at" + (i - m);
```

---

Shift:

valid shift will always occurs one before starting index & i - m gives that

BHATTI

CamScanner

ate

## Run it on text.

After we have made the FA now run it on the text using the pseudocode

T = ababababacaba

P = abababaca

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| T = | a | b | a | b | a | b | a | c | a | b | a |
| next state: | 1 | 2 | 3 | 4 | 5 | 4 | 5 | 6 | 7 | 2 | 3 |

q == m ✓ (for the first time)

shift = i − m

= 9 − 7 = 2

valid shift = 2.

BHATTI

CamScanner

y/date

Complexity:

Preprocessing: Building transition table: ($\delta$)

$= \Theta(m^3 \cdot |\Sigma|)$

This is due to naive approach.

Matching time: $\Theta(n)$ only.

Pseudocode for naive building of ($\delta$):

Compute-Transition-Function ($P, \Sigma, m$):

for $q = 0$ to $m$

for each character $a \in \Sigma$

$k = \min\{m, q+1\}$

while $P[:k]$ is not a suffix of $P[:q]a$

$k = k - 1$

$\delta(q, a) = k$

return $\delta$

But this is very slow, to make it faster we have KMP.

This approach also need space of: $O(m \cdot |\Sigma|)$

BHATTI

CamScanner

ate

## 4) Knuth • Morris & Pratt (KMP) algorithm

This is a follow up from the automata approach.

It achieves linear time ($O(n)$) matching without generating a full $O(m \cdot |\Sigma|)$ table like FA.

KMP uses a simple prefix Function ($\pi$ Table) of size $m$ that tells us exact shift to make whenever a mismatch occurs.

Prefix function + Matching function.

BHATTI

CamScanner

/date

# Prefix function / table:

Like we did in FA, we see for the pattern and its length, we need to find the length at every size of pattern & that makes the $\pi$ table.

e.g.:

P: aabaab

| length | Prefixes | Suffixes | longest equal length |
|---|---|---|---|
| 1- a | ∅ | ∅ | 0 |
| 2- aa | a | a | 1 |
| 3- aab | a, aa | b, ab | 0 |
| 4- aaba | a, aa, aab | a, ba, aba | 1 |
| 5- aabaab | a, aa, aab, aaba | b, ab, baab, abab | 0 |

$\pi$ = {0, 1, 0, 1, 0}

BHATTI

CamScanner

These longest lengths form the $\pi$ table.

Prefix function pseudocode:

```
Compute-Prefix-function (P)
    m = P.length
    π = new [1...m] size array
    π[1] = 0
    k = 0
    for q = 2 to m                    # start from 2 because π[1] = 0 already
        while k > 0 and P[k+1] != P[q]
            k = π[k]                  # fallback to previous longest prefix
        if P[k+1] == P[q]
            k = k+1
        π[q] = k
    return π
```

BHATTI

CamScanner

late

KMP - Matcher - Algorithm:

```
KMP-MATCHER(T, P)
    n = T.length
    m = P.length
    π = Compute_prefix();

    q = 0

    for i = 1 to n
        while q > 0 and P[q+1] ≠ T[i]
            q = π[q]   # not matched & but
                       # our pattern may partially
                         found
                       so go back to the
                       previous longest
    if P[q+1] == T[i] then
        q = q+1

    if q == m then   # pattern found
        print pattern at shift i-m.
        q = π[q].   # fall back as well
```

BHATTI

CamScanner

//date

Complexity:

Preprocessing time: $\theta(m)$ to build $\pi$

Matching time: $\theta(n)$, but how we have nested loop?
In actuality, the while inside for rarely runs so it becomes $\theta(n)$ only.

Total time complexity = $\theta(n+m)$

Space complexity = $\theta(m)$ to store $\pi$-array.

BHATTI

CamScanner

ate

Example:

T = acbaabab
P = aababab

will do this after & show example.

$\pi$ = [0, 1, 0, 1, 0] ← needs to be computed, I just computed it for eg.

| i | q | P[q+1] T[i] | q == m |
|---|---|---|---|
| 1 | 0 | P[1] = a ⟷ a = T[1] (same, so q = q+1) | 1 ≠ 5 |
| 2 | 1 | P[2] = a ⟷ a = T[2] (same, so q = q+1) | 2 ≠ 5 |
| 3 | 2 | P[3] = b ⟷ b = T[3] (same, so q = q+1) | 3 ≠ 5 |
| 4 | 3 | P[4] = a, a = T[4] | 4 ≠ 5 |
| 5 | 4 | P[5] = b ≠ a = T[5] (not same so run the while loop) | 2 ≠ 5 |
| | | q = 2 | |
| 6 | 2 | P[3] = b, b = T[6] | 3 ≠ 5 |
| | | q+1 = 3 | |

BHATTI

CamScanner

ay/date

| | | | |
|---|---|---|---|
| 7 | 3 | P[4] a $\xrightarrow{\text{same}}$ a T[7] (q=4) | 4 ≠ 8 |
| 8 | 4 | P[5] b $\xrightarrow{\text{same}}$ b T[8] (q=5) | 5 == 5 ✓ |

True so print the valid shift.

BHATTI

CamScanner

ate

Prefix table finding example using pseudocode

P = aabaab

| q | P[q] | current k | while loop | k after loop | IF P[k+1] == P(k) | Final k | π[q] |
|---|---|---|---|---|---|---|---|
| 1 | a | – | ignored – as | | loop starts from 2 | | 2 – 0 |
| 2 | a | 0 | ✓ runs | 0 | ✓ runs P[1] == P(2) same so k = k+1 | 1 | 1 |
| 3 | b | 1 | ✓ runs & as P(2) ≠ P(3) so k=0 | 0 | P(1) X P(3) ✗ not same | 0 | 0 |
| 4 | a | 0 | ✗ | 0 | ✓ runs | 1 | 1 |
| 5 | b | 1 | ✓ runs | 0 | ✗ | 0 | 0 |

So π = [0, 1, 0, 1, 0]

BHATTI

CamScanner